

Introduction to Computer Networking

Mail Server Project Report

Romain Malaise, Baptiste Renard
Students ID: s2300791, s2302437

December 14, 2025

1 Introduction

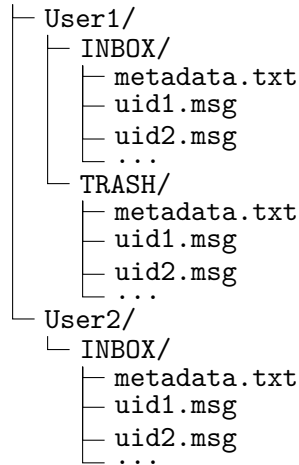
For the course of introduction to Computer Networking, we implemented a mail server using Java Sockets. The server uses the SMTP, POP3 and IMAP protocols to allow sending, storing and retrieving emails. We use a simulated Docker network composed of three domains. Each domain has its own DNS and Mail Server. We also have to handle SMTP forwarding between domains and to implement persistent message storage. Therefore, we developed a mail server in Java using a OOP approach which allowed us to answer all the requirements of the assignment.

2 Software Architecture

The program is divided into 13 classes, each with a unique role:

- **MailServer:** Main class of the mail server. Its role is to create one thread per protocol (SMTP, POP3, IMAP), each with its own ServerSocket. Each ServerSocket listens for incoming client connections and accepts them (putting them in a Thread Pool). For every new client connection, it creates client Socket and a ClientHandler corresponding to the protocol of the listening socket and submits it to the Thread pool.
- **ClientHandler:** creates the Handler corresponding to the protocol used and starts handling the session through handleSession method of Handler. Moreover, it closes the connection and frees the resources.
- **Handler:** abstract class defining the global behaviour of each protocol Handler. It defines the handleSession method, which defines the exact behaviour of a discussion Client-Server. Therefore, this abstract class ensures that only two specific methods related to each protocol need to be implemented for the correct execution of handleSession. These two methods are sendGreeting and handleCommand, modeling the only part that changes depending on the protocol used.
- **SMTPHandler/POP3Handler/IMAPHandler:** Subclasses of Handler to implement the two abstract methods needed. They implement all commands required by each protocol and ensure that each command is correctly parsed and validated. They make the command action and response to the client.
- **Message:** A class to represent a mail. Handle all protocols and by consequence, its attributes are the following : uid, flags, recipients, sender, content.
- **Mailbox:** A class to model a mailbox, containing its messages, nextUid given to the next message it receives, its UIDVALIDITY and its name (e.g. INBOX, WAITING,...).

- **MailStore:** This class allows us to store Message on disk. We note that each Message is encapsulated in a Mailbox object. The class MailStore is not instantiated. It only contains static method to manage the disk. It implements methods such as loadMailbox, saveMailbox, createMailBox, expunge, ... It is useful to load Message / Mailbox, or save modifications of a Mailbox on the disk. The tree structure of the storage of messages and mailboxes are the following :



In each mailbox folder, we have a metadata file and a file for each message. This structure is a good way to solve the problem of message persistence, encapsulating the whole content of a mail in a single file. These files are structured in the following way :

```

metadata.txt
  UIDVALIDITY: <unique integer>
  NEXTUID: <integer>
  <uidMsg1> <flag1> <flag2> ... <size>
  <uidMsg2> <flag1> <flag2> ... <size>
  ...

```

```

.msg
  FROM: <from>
  RCPTS: <rcpt1> <rcpt2> ...
  SUBJECT: <subject>

  <body line 1>
  <body line 2>
  <body line 3>
  ...

```

- **User:** A class to model a user. It is simply a user characterized by a email address, but it also contains useful methods to check if the domain corresponds to the current Mail Server, the password, if user exists, ...
- **SMTPTransmitter:** This class is used for SMTP forwarding. It is instantiated when a recipient does not match the current Mail Server. In this case, we have to forward the mail to another server and we fulfil this by calling the forward method of SMTPTransmitter.
- **DomainResolver:** A class implementing the resolve method to resolve a domain with a chosen type record (A or MX). And a more specific method resolveSmtpServer used by the SMTP forwarder.

- **ConnectionIO:** Reading and sending lines through the session socket. It automatically appends the empty line terminator, writes, and flushes the data. It also allows us to avoid redundancy when instantiate a `BufferedReader` or `OutputStream`. Moreover, it has a method to correctly free the ressources and close the connection. Handle IO exception, `SocketTimeout`,... too.

This structure in classes of the project ensures a good modularity, readability and clarity.

3 Mail forwarding and resolution

First, the SMTP client sends the mail to the mail server of gembloux `mail.gembloux.uliege.be`. It compares the domain of the recipient with its own. It detects that `info.uliege.be` is not a local user for `gembloux.uliege.be`. Then, it resolves `info.uliege.be` through its DNS server and forwards the mail by SMTP connection to `mail.uliege.be`. (`info.uliege.be. IN MX 10 mail.uliege.be.`), as we said that the recipient is not a local user. The server `mail.uliege.be` receives the mail by SMTP and compares the recipient to its own domain. It does not match, so it resolves the mail server for `info.uliege.be` through its DNS server, and finds `mail.uliege.be`. From the point of view of the mail server of `uliege.be`, it is the final destination and it does not forward any more and stores the mail on its own server. The mail does not reach `mail.info.uliege.be` and is stored on `mail.uliege.be`, playing the role of a central server. Therefore, the mail never reaches `mail.info.uliege.be`.

4 Multi-thread coordination

First, we use a fixed-size thread pool to handle incoming SMTP, POP3 and IMAP connections, as suggested in the instructions. This allows us to avoid creating one thread per connection, which prevents DoS issues. Then we also ensure that the server can keep simultaneously listening for new connections on each server socket by launching them in three dedicated threads. Furthermore, the thread pool executes each handler task (`ClientHandler`) in separate threads, allowing multiple clients connections to be handled in parallel. There is no shared memory between handlers, reducing the fact that we have to handle synchronization. However, concurrent IMAP clients on the same mailbox could overwrite each other's changes. For example, two clients may load the same mailbox into two different `Mailbox` instances. If one modifies and saves the mailbox before the other, the second save will overwrite the first changes. To solve this problem, one could use a lock per user to prevent these race conditions.

5 Limits

First limit is the mailbox concurrency. It is not fully synchronized when multiple IMAP clients access the same mailbox at the same time. Then, there is no protection against partial writes or file corruption. For example if an error occurs during writing, an expunge could have changed the metadata file of a mailbox (removing the presence of the mail in the metadata) but not yet remove the message file on the disk. Furthermore, we have implemented a subset of all existing SMTP, POP3, IMAP commands. To finish, our protocols do not use TLS or encryption.

6 Possible Improvements

A major improvement is to make the mailbox concurrent with multiple clients accessing the same mailbox. We could have fix this problem using locks, but we did not have the time needed to achieve this. Another improvement could be to store passwords hashed instead of clean text. Furthermore, we could also replace our storage by a real database backend. On the side of user

experience improvements, we could make some logs handling. For the other improvements, they are solving problems mentioned in limits. Finally, we could expand the functions of the IMAP UID FETCH command with filters as in described in the protocol reference.

7 Conclusion

First, we faced many challenges in this project. In fact, it was difficult to understand the whole assignment. There was a lot of information to know and to reflect on before starting to code. For example, we had to understand how to handle the persistent mail storage. We also had to understand SMTP forwarding. But this was not all: the protocol implementations included many details such as UID, UIDVALIDITY, FLAGS, EXPUNGE, ... There were many notions to master before coding, in order to concept a good organization of the classes and achieve the project. To conclude, we learned many things, such as how real mail protocols (SMTP, POP3, IMAP) work, the role of DNS in routing emails, SMTP forwarding and software architecture (different layers handled in separate classes). Furthermore, we must not forget that this project also helped us improve our Java skills.